18 May 2026

Continuity: Persistent Decision-Centric AI Cognition with Runtime Governance, Bounded Retrieval, and Defense-in-Depth Credential Scrubbing — Defensive Technical Disclosure v2.0

Thiago Goncalves
Hackerware LLC

Continuity — Defensive Technical Disclosure v2.0

Thiago Goncalves

2026-05-17

CONTINUITY: PERSISTENT DECISION-CENTRIC AI COGNITION WITH RUNTIME GOVERNANCE, BOUNDED RETRIEVAL, AND DEFENSE-IN-DEPTH CREDENTIAL SCRUBBING

DEFENSIVE TECHNICAL DISCLOSURE — Version 2.0

Field	Value
Author	Thiago Goncalves (alienfader)
Organization	Hackerware LLC
Date of Publication	May 17, 2026
Earliest Engineering Commit	October 3, 2025
Earliest Public Release	October 31, 2025 (Visual Studio Code Marketplace)
Supersedes	`Continuity_Defensive_Publication_Full.pdf` (March 26, 2026)
Public timestamp venues	(1) GitHub release tag in the public repository `Thiagoscode/continuity-defensive-publication`; (2) Technical Disclosure Commons submission at `tdcommons.org` (Elsevier-operated, indexed by USPTO and global patent offices)
Identifier	Continuity-DP-v2.0-2026-05-17

NOTICE OF PUBLIC DISCLOSURE

This document is a defensive technical publication. Its purpose is to place specific technical methods, architectures, parameter values, threshold calibrations, and feature combinations into the public prior-art record so that no party other than the author may obtain a patent on the same combinations and assert such a patent against the author or against third parties practicing the disclosed methods.

The author hereby publicly discloses the methods described below and declines patent assertion rights over the specific combinations disclosed as of the date of publication. Source code remains proprietary under the author's existing licensing; this disclosure covers METHODS and ARCHITECTURES, not implementation source.

This document does NOT claim invention of every component described. Where the author is aware of related prior art predating this disclosure or the underlying Continuity work, that prior art is explicitly acknowledged in Section II.D. The disclosure is of specific COMBINATIONS, parameter choices, and named threat models, not of the underlying generic methods.

ABSTRACT

A memory-augmented AI assistant control plane is disclosed, comprising five technical methods combined and publicly released as a Visual Studio Code extension since October 31, 2025: (A) **bounded retrieval** with a code-enforced per-call cap and per-record character limits such that worst-case per-turn token injection remains approximately constant regardless of total stored knowledge; (B) **runtime governance interception** of AI agent tool calls using an INTERCEPT → PAUSE → EVALUATE → ENFORCE pipeline mapped to nine OWASP LLM Top 10 (2025) categories; (C) **defense-in-depth credential scrubbing** applied at five enforcement boundaries (tool-call input, AI-generated output, persistence write, persistence read, MCP tool-result) using twenty-seven provider-specific regex patterns and a Shannon-entropy fallback empirically calibrated to 4.5 bits per character; (D) **multi-signal relationship inference** between architectural decisions combining semantic, temporal, and tag-set signals, with **Markov chain prediction** of next-decision topics from tag-set transitions; and (E) an extended threat model naming a **"Memory Amplifier"** attack class unique to memory-augmented AI assistants, in which a one-shot credential exposure persists in a memory store and is re-injected into the model context on every subsequent session.

I. FIELD OF DISCLOSURE

The disclosure relates to memory-augmented artificial intelligence assistants used in software engineering workflows, including:

- Integrated development environment plugins coupled with AI coding assistants via the Model Context Protocol (MCP)
- AI chatbots offering persistent memory across sessions
- Autonomous agent frameworks with persistent scratchpads or memory stores
- Retrieval-Augmented Generation (RAG) systems whose retrieval index may contain inadvertently-captured sensitive content
- Architectural Decision Record (ADR) systems with automatic relationship inference
- Any AI-assisted system that writes a portion of one session's context to a persistent store for retrieval into future sessions

---

II. BACKGROUND AND PROBLEM STATEMENT

II.A. The Memory Wall in static-embedding context approaches

Conventional approaches to providing AI assistants with persistent context — static embedding of architectural decisions or notes into context files such as `CLAUDE.md`, `.cursorrules`, `AGENTS.md`, `GEMINI.md`, `.github/copilot-instructions.md` — produce linear token growth as the decision corpus accumulates:

> **$T_{static}(n) = b + n \cdot \bar{d}$**

where: - n is the total number of stored decisions - $\bar{d}$ is the average tokens per decision (measured at the author's corpus: $\bar{d} \approx 222$ tokens per decision, derived from an average of 887 characters per decision divided by 4 characters per token under the GPT-style byte-pair-encoding tokenization convention) - b is the base context tokens consumed by the AI assistant's system instructions

At a typical 200,000-token context window, overflow occurs at approximately:

> **$n \approx 902$** decisions (without base instructions subtracted) **$n \approx 870$** decisions (with a measured base of ~7,053 tokens subtracted)

At the author's actual measured corpus of n = 2,200 decisions as of the publication date (2026-05-17), static embedding would require approximately 488,400 tokens — more than 2.4× a typical large-context window. The conventional approach defeats itself as knowledge accumulates. This is the "Memory Wall."

II.B. The Compromised Context threat model

AI assistants in 2024-2026 increasingly execute tools (file operations, network calls, code execution, message sending) on behalf of users. Without runtime governance, prompt injection — direct or indirect — can manipulate the assistant into emitting tool calls that exfiltrate data, modify state, or perform unauthorized actions.

Foundational published work in this area includes Greshake et al. (2023), "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." The ClawdBot incident — later renamed OpenClaw and then Moltbot following Anthropic's trademark request on January 27, 2026; assigned CVE-2026-25253 (CVSS 8.8); documented contemporaneously by Snyk, VentureBeat, ByteVanguard, and Kaspersky — demonstrated practical exploitation against an autonomous agent in January 2026.

II.C. The Memory Amplifier threat (named extension)

A third problem, specific to memory-augmented AI assistants, is disclosed and named herein the **"Memory Amplifier" threat**: a one-shot credential exposure or content leak becomes a permanent re-exposure vector via the persistent memory layer. The threat unfolds as follows:

1. A credential or sensitive string enters the AI context once (paste by user, IDE selection injection, file-content injection, retrieved-document carryover, command-line argument forwarding)
2. The AI assistant produces output incorporating the sensitive string
3. A memory-augmenting system writes some portion of the session — output, summary, retrieved documents, or even raw input — to a persisted store (vector database, JSON file, SQL row, embedding index)
4. Every subsequent AI-context turn that retrieves from this store re-injects the sensitive string
5. Probability of re-exposure scales with retrieval frequency and corpus growth, not with operator vigilance
6. For append-only or unbounded-retention memory stores, the probability collapses to "indefinitely." For TTL-based or similarity-retrieved stores, it remains probabilistic but unbounded compared to one-shot leakage.

Concrete reproducer (May 2026): An IDE harness's selection-injection feature forwarded an editor selection containing a real Atlassian API token (prefix `ATAT…`; full value redacted and rotated upon detection) to an AI assistant. The AI assistant produced commentary that echoed token fragments **twice** during the session — once in immediate analysis, and once again from a persisted summary in a subsequent turn. Without persistence-layer scrubbing, the token would have re-injected on every future session.

The threat is structural to memory-augmented AI. It is **not adequately addressed by**: - Input scanning alone (does not catch credentials introduced via file-content injection or retrieval carryover) - Output filtering alone (does not address credentials persisted *before* the filter runs) - Single-boundary vector-store sanitization (other write paths bypass it) - OS keychain credential storage alone (eliminates source but does not retroactively scrub historical context)

A defense-in-depth approach is required because the layers fail independently.

II.D. Explicit acknowledgment of related prior art

This disclosure is made with awareness of related published or commercial prior art and explicitly does **NOT** claim invention of:

Prior art	Notes
AI memory in general	Mem0 (2024-2025, arxiv:2504.19413); MemGPT/Letta (October 2023, arxiv:2310.08560); ChatGPT Memory; LangChain memory modules
Top-K retrieval bounds in RAG	OpenAI Assistants `file_search` (April 2024) with server-enforced `max_num_results ≤ 50` and ~16K token budget; Azure AI Search

Prior art	Notes
	semantic ranker (November 2024) with per-summary 2,048-token cap; Pinecone, Weaviate, Chroma, Qdrant default top-K semantics
Identical retrieval count	Mem0 (April 2025) uses retrieval count `s = 10`, identical to the value disclosed below
Runtime LLM guardrails (output filtering)	NeMo Guardrails (October 2023); AWS Bedrock Guardrails (April 2024 GA); Anthropic Constitutional AI; Lakera Guard; Cloudflare AI Gateway Guardrails (February 2025); Microsoft Azure Prompt Shields (April 2024, with direct-vs-indirect-prompt-injection distinction); GuardAgent (June 2024)
MCP / agent tool-call interception	Invariant Labs Guardrails (April 2025; acquired by Snyk June 2025) — transparent MCP/LLM proxy with explicit argument matching and data-flow tracking; OpenAI Agents SDK tripwires (March 2025); Strands `before_tool_call` hook; Progent (April 2025); AgentSpec (March 2025)
Pre-OS syscall filtering for AI agents	IsolateGPT/SecGPT (March 2024) using `seccomp` to filter syscalls at the OS boundary
Generic secret/credential scanning	Yelp `detect-secrets` (2018) — **default `Base64HighEntropyString` threshold is 4.5 bits per character**, ships ~27 detectors; TruffleHog (entropy-based, ~800+ patterns); GitGuardian (commercial, 350+); Gitleaks (regex); GitHub secret scanning (Microsoft); AWS Macie; CyberArk US Patent 11,550,569 B2
LLM-aware credential redaction	LLM Guard (laiyer-ai/llm-guard, 2023) — already deploys `detect-secrets` at the LLM input AND output boundaries (2-boundary deployment); Lakera secret detection; arxiv:2511.20920 (November 2025) recommending "post-call interceptors handle redaction and PII scrubbing" in MCP pipelines; scrubfile (PyPI 2024-25) for MCP tool-response scrubbing
Architectural Decision Records (ADRs)	ADR-tools by Nat Pryce (github.com/npryce/adr-tools); MADR; Log4brains; Structurizr; Backstage ADR plugin; ADR Manager VS Code extension; ADDSS; PAKME; ADkwik; AWS

Prior art	Notes
	Prescriptive Guidance for ADRs. Every surveyed ADR tool requires **manual** relationship entry (e.g., `[supersedes: ADR-001]`).
Markov chains in software tooling	Hindle, Barr, Gabel, Su, Devanbu (2012) "On the Naturalness of Software" (ICSE) — n-gram Markov models applied to lexical code token autocomplete. Different state space (lexical tokens) than disclosed below (decision tag sets).
Decision-relationship inference from NLP	Kantara (Soliman et al., 2023-2025, arxiv:2311.03358 and arxiv:2506.11005) — auto-infers similarity and contradiction between developer-rationale decisions in commit messages using DistilBERT and RoBERTa-MNLI. Single-signal NLP; 2 relation types; commit messages as input.
OWASP LLM Top 10 framework	OWASP LLM01:2025–LLM10:2025; OWASP MCP01:2025 (Token Mismanagement and Secret Exposure, published 2025); OWASP ASI06:2025 (Memory & Context Poisoning, published December 9, 2025) — names the persistent memory threat class but from an attacker-injection perspective, not the user-paste amplification perspective disclosed herein
AI memory attack research	Rehberger "SpAIware" (September 2024); MINJA; RAGPoison; Promptware; AI Recommendation Poisoning

The specific combinations, parameter choices, and named threat models disclosed below are intended for the prior-art record. Where the disclosure overlaps with prior art, it should be read as confirming public knowledge rather than asserting new invention.

III. DEFINITIONS

Term	Definition
AI Assistant	A software agent backed by a large language model or similar generative model that receives input context, produces output, and may invoke external tools

Term	Definition
Memory-Augmented AI Assistant	An AI assistant whose operation includes one or more persisted stores from which content is read into AI context across sessions, including but not limited to RAG retrieval indices, decision logs, scratchpads, vector databases, and Architectural Decision Record stores
Decision Record	A structured record of an engineering choice and its reasoning, comprising at minimum a question, an answer, and a tag set
Decision Corpus	The set of all decision records persisted for a single project or workspace
Bounded Retrieval	Retrieval logic that maintains an upper bound on injected context size such that the bound does not grow with the size of the decision corpus
Governance Lock	A runtime enforcement layer that evaluates proposed AI assistant tool calls against policy and prior decisions before the call reaches the operating system
Memory Amplifier Threat	The threat class wherein a credential or sensitive string persisted in a memory-augmented AI store is re-injected into AI context across subsequent sessions, multiplying exposure surface
Credential	A high-confidence sensitive string identifying or authorizing access, including but not limited to API keys, OAuth tokens, personal access tokens, private keys, JWTs, session tokens, database connection strings, and password values
Provider-Specific Pattern	A regular expression matching a known-format credential prefix issued by an identified service provider (e.g., `ghp_*` for GitHub, `sk-ant-*` for Anthropic)
Entropy Fallback	A Shannon-entropy-based detector for high-entropy candidate strings lacking a recognized provider prefix
Scrub	The act of replacing a detected credential in a text body with a redaction sentinel
Idempotent Scrub	A scrub function for which `scrub(scrub(x)) === scrub(x)`
Boundary	

Term	Definition
	A point in the AI assistant data flow at which an enforcement check is applied
Model Context Protocol (MCP)	An open protocol enabling AI assistants to access external tools, read resources, and interact with systems

IV. SYSTEM ARCHITECTURE OVERVIEW

Continuity operates as an intermediary cognition and control layer between the developer's IDE / AI assistant and the underlying language model and tooling environment. The system is not itself the language model. The high-level data flow:

```
Developer
   │
   ▼
IDE / AI Assistant
   │
   ▼
Continuity Extension + MCP Server
   │
   ├─ Decision capture, ranking, injection (Method A)
   ├─ Tool-call governance interception (Method B)
   ├─ Five-boundary credential scrubbing (Method C)
   └─ Multi-signal relationship inference + Markov prediction
(Method D)
   │
   ▼
Persistent memory store (.continuity/decisions.json, embeddings,
session notes)
   │
   ▼
Future AI session retrieves bounded context
```

V. METHOD A — BOUNDED RETRIEVAL ARCHITECTURE

V.A. Method summary

Per-turn token consumption is decoupled from the total number of stored decisions by enforcing:

1. A code-level hard ceiling on retrieval count
2. Per-record character caps applied to both the question and answer fields of each decision record
3. A closed-form worst-case token bound stated as a design invariant

V.B. Specific parameter values disclosed

The following constants are disclosed to the public record:

Constant	Value	Description
`TOP_K_CEILING`	**10**	Hard maximum number of decisions returned per retrieval call, enforced in code at the retrieval layer. Requested values exceeding this ceiling are clamped down to 10.
`DEFAULT_MAX_DECISIONS`	**3**	Default retrieval count when caller does not specify
`MAX_TEXT_LENGTH`	**2,000**	Maximum characters retained from a decision's question field for injection
`MAX_ANSWER_LENGTH`	**5,000**	Maximum characters retained from a decision's answer field for injection

V.C. Closed-form worst-case bound

The worst-case per-turn token injection is stated as a design invariant:

> **T_turn ≤ TOP_K_CEILING · (MAX_TEXT_LENGTH + MAX_ANSWER_LENGTH) / 4**
>
> **= 10 · (2,000 + 5,000) / 4**
>
> **= 17,500 tokens**

This bound is **independent of n**, the total number of stored decisions. The bound does not grow as the decision corpus accumulates. This is the distinguishing property versus static-embedding approaches whose token cost grows as $T_{static}(n) = b + n \cdot \bar{d}$.

V.D. Measured reduction

At the author's measured corpus of n = 2,200 decisions (as of publication date 2026-05-17):

- Static embedding would require: 2,200 × 222 = **488,400 tokens**
- Bounded retrieval requires: ≤ 17,500 tokens (worst case)
- **Reduction: ~96.4%**

At n = 795 (earlier measured snapshot):

- Static embedding: 795 × 222 = 176,490 tokens

- Bounded retrieval: ≤ 17,500 tokens
- **Reduction: ~90.1%**

The reduction percentage *grows* with corpus size — the larger the project history, the greater the benefit. This is structurally opposite to static embedding, which becomes more expensive as knowledge accumulates.

V.E. Schema specificity

The disclosed bound applies to a structured `{question, answer, tags, metadata}` record schema, not to generic chunked text. The Q/A schema enables per-field caps because each field has bounded semantic role. Chunked-text approaches cannot apply field-specific caps because chunks lack semantic structure.

V.F. Retrieval pipeline disclosed

```
Decision corpus (.continuity/decisions.json)
   │
   ▼
Hybrid candidate retrieval (keyword + semantic)
   │
   ▼
Multi-signal ranking (quality, recency, relationship, staleness)
   │
   ▼
Top-K selection with K ≤ TOP_K_CEILING (clamp enforced)
   │
   ▼
Per-record truncation: question ≤ MAX_TEXT_LENGTH, answer ≤
MAX_ANSWER_LENGTH
   │
   ▼
Bounded injection into AI context (≤ 17,500 tokens worst case)
```

VI. METHOD B — RUNTIME GOVERNANCE LOCK

VI.A. Method summary

AI assistant tool calls are intercepted at the protocol layer and evaluated against a policy mapped to OWASP LLM Top 10 (2025) categories before the corresponding system action is permitted. The pipeline:

> **INTERCEPT → PAUSE → EVALUATE → ENFORCE**

The disclosed property: when policy evaluation fails, the operation **does not reach** the operating system. The interceptor returns a denial response to the AI assistant. The OS never receives the corresponding syscall.

VI.B. Acknowledgment

This broad concept overlaps substantially with published prior art including Invariant Labs Guardrails (April 2025), IsolateGPT/SecGPT (March 2024) using seccomp, NeMo Guardrails (2023), AWS Bedrock Guardrails (2024), Microsoft Azure Prompt Shields (April 2024), Lakera Guard, Cloudflare AI Gateway Guardrails (February 2025), and the OpenAI Agents SDK tripwires (March 2025). The author does not claim invention of pre-execution AI tool-call governance.

This section discloses the specific Continuity implementation for the prior-art record.

VI.C. OWASP LLM Top 10 (2025) coverage disclosed

The Continuity policy evaluates tool calls against at least nine OWASP LLM Top 10 (2025) categories. The 2025 edition renumbering and renaming versus the 2023 v1.1 is preserved:

OWASP 2025 ID	Name	Continuity policy action
LLM01:2025	Prompt Injection	BLOCK
LLM02:2025	Sensitive Information Disclosure	SANITIZE (via Method C)
LLM04:2025	Data and Model Poisoning	SANITIZE
LLM05:2025	Improper Output Handling	BLOCK
LLM06:2025	Excessive Agency	INTERCEPT
LLM07:2025	System Prompt Leakage	SANITIZE (via Method C)
LLM08:2025	Vector and Embedding Weaknesses	SANITIZE
LLM09:2025	Misinformation	WARN
LLM10:2025	Unbounded Consumption	LIMIT

VI.D. Source-aware policy

Tool-call arguments are tagged with their source provenance (user-typed terminal input, AI-retrieved content from persistent memory, external-tool output). Policy decisions are conditioned on source: arguments derived from retrieved memory or external tool output receive stricter scrutiny than user-typed input, because the former are vectors for indirect prompt injection and Memory Amplifier exposure.

VI.E. Outcome categories

For each evaluated tool call, the Governance Lock returns one of: `allow`, `warn`, `block`. The mode (warn-versus-strict) is configurable. In strict mode, `block` prevents execution. In warn mode, `block`-classified calls execute but are flagged in the response.

VI.F. Audit logging

Every intercepted call is logged with: tool name, sanitized argument summary, evaluated policies, outcome, and timestamp. The log is itself subject to Method C scrubbing at the persistence boundary.

VII. METHOD C — DEFENSE-IN-DEPTH CREDENTIAL SCRUBBING

VII.A. Method summary

Credentials and other high-confidence sensitive strings are scrubbed at **five enforcement boundaries** in the memory-augmented AI assistant pipeline. A shared scrub primitive is invoked at each boundary, providing defense-in-depth: any single layer's failure leaves four others.

VII.B. The five boundaries disclosed

Boundary	Description	Implementation location (preferred embodiment)
1. Input	MCP tool-call arguments scrubbed before tool handlers execute	Pre-handler middleware on the MCP request path
2. AI output	AI-generated text scrubbed at the chokepoints where it is consumed by downstream code	Wrapper on `LLMProviderManager.generateDecision()` and `AnthropicAPILayer.parseResponse()`
3. Persistence write	Content scrubbed before persisting to the memory-augmented store	Wrapper on `DecisionCRUD` write paths, session-notes appender, working-memory service, handoff-artifacts service, project-instructions generator
4. Persistence read	Content scrubbed on retrieval from the memory-augmented store, as defense-in-	Wrapper on `DecisionCRUD` read paths

Boundary	Description	Implementation location (preferred embodiment)
	depth against (a) content persisted before boundary 3 was deployed, (b) content written via an alternative path that bypassed boundary 3, (c) tampered store content	
5. MCP tool-result	MCP tool response payloads scrubbed post-handler, before return to the AI assistant	Post-handler middleware on the MCP response path; composed dispatcher: `scrubToolResponse` followed by `attachCredentialWarnings`

VII.C. The 27 provider-specific patterns disclosed

The shared scrub library implements a catalog of provider-specific high-confidence patterns. The catalog is publicly disclosed:

#	Provider / Family	Pattern prefix(es)
1	GitHub classic PAT	`ghp_*`
2	GitHub OAuth	`gho_*`
3	GitHub server token	`ghs_*`
4	GitHub user token	`ghu_*`
5	GitHub refresh token	`ghr_*`
6	GitHub fine-grained PAT	`github_pat_*`
7	GitLab PAT	`glpat-*`
8	Atlassian (Jira, Confluence)	`ATATT3xFfGF0…`
9	OpenAI standard	`sk-*`
10	OpenAI project	`sk-proj-*`
11	Anthropic	`sk-ant-*`
12	Slack family	`xox[baprso]-*` covering `xoxb-`, `xoxa-`, `xoxp-`, `xoxr-`, `xoxs-`, `xoxo-`
13	AWS access key	`AKIA*`

#	Provider / Family	Pattern prefix(es)
14	AWS temporary access key	`ASIA*`
15	Google API key	`AIza*`
16	Stripe live secret	`sk_live_*`
17	Stripe test secret	`sk_test_*`
18	Stripe restricted	`rk_*`
19	Twilio SID	`SK*`
20	Twilio Account SID	`AC*`
21	SendGrid	`SG.*`
22	Mailgun	`key-*`
23	NVIDIA API	`nvapi-*`
24	Perplexity	`pplx-*`
25	JSON Web Token (JWT) structural	`eyJ*.*.*` — three base64url segments separated by `.`
26	PEM private key block	`-----BEGIN <KIND> PRIVATE KEY-----`
27	Environment-style assignment	`NAME=VALUE` where NAME matches credential-suggestive patterns and VALUE has sufficient entropy

The catalog is extensible at runtime. Coverage of further provider prefixes (e.g., Slack `xoxe-`, `xapp-`, `xwfp-`; future-issued providers) can be added without altering the five-boundary architecture.

VII.D. Shannon-entropy fallback

For strings that do not match any provider-specific prefix, a Shannon-entropy fallback applies to candidate substrings:

- **Size range:** 32 to 512 characters
- **Calculation:** `H(x) = −Σ p_i log_2 p_i` over the character distribution of the candidate
- **Threshold:** strings with `H(x) ≥ 4.5 bits per character` are flagged

Disclosure: the 4.5 bits-per-character threshold value is NOT claimed as novel. Yelp's `detect-secrets` library has used 4.5 as its default `Base64HighEntropyString` threshold since at least 2018. What IS disclosed is the specific calibration evidence:

Threshold	False-positive rate on author's ~1,200-string negative corpus
3.5 bits/char	~82%
4.0 bits/char	~27%

Threshold	False-positive rate on author's ~1,200-string negative corpus
4.5 bits/char (default)	**0%**
5.0+ bits/char	0%

The negative corpus comprises UUIDs, SHA-256 / MD5 hashes, git SHAs, file paths, log lines, trace IDs, English prose, TypeScript source code, JSON configuration, and base64-encoded image blobs.

The positive corpus comprises 53 samples covering all 27 pattern categories. Measured recall: 53/53 = 100% at default settings. Measured false positives: 0/11 buckets on the negative corpus.

VII.E. Overlap deduplication

When a provider-specific pattern and the entropy fallback both match overlapping spans, the provider-specific match wins and the entropy match is suppressed. This prevents double-redaction.

VII.F. Confidence tagging

Each detection is tagged with a confidence level: `high` (provider-specific match) or `medium` (entropy-only match). The default policy is `redact-high, flag-medium`. An opt-in policy redacts both.

VII.G. Idempotency property

The scrub function satisfies:

> **scrub(scrub(x)) === scrub(x)**

This property is achieved by replacing detected credentials with redaction sentinels of the form `<REDACTED:pattern-name>` (e.g., `<REDACTED:github-pat-classic>`) that: 1. Do not match any provider-specific pattern in the catalog 2. Have insufficient Shannon entropy to trigger the fallback detector

Idempotency allows safe application of the scrub primitive at multiple boundaries without compounding redactions or producing degenerate output.

VII.H. `_meta.credential_warnings` protocol-level signaling

Detected credentials at boundary 1 (input) are attached to the MCP response under a `_meta.credential_warnings` field:

```
{
  "result": { ... },
  "_meta": {
    "credential_warnings": [
      {
        "tool": "log_decision",
        "argument_path": "answer",
```

```
        "pattern": "github-pat-classic"
      },
      {
        "tool": "log_decision",
        "argument_path": "question",
        "pattern": "entropy"
      }
    ]
  }
}
```

Each warning identifies the tool name, the JSON path within the argument where the credential was found, and the matched pattern name. The matched secret itself is not included. Confidence level is implicit: only high-confidence (provider-pattern) matches and entropy-fallback matches are surfaced via this channel — by default only high-confidence matches trigger redaction. Output-side and tool-result scrubbing (boundary 5) redact silently without emitting this metadata; the `audit_secrets` tool is the explicit mechanism for surfacing tool-result detections to the user.

VII.I. `audit_secrets` MCP tool

The system exposes an `audit_secrets` MCP tool that scans the memory-augmented store, enumerates detected credentials with their locations and matched pattern names, and reports to the user. The tool is itself **allowlisted out** of the boundary-5 tool-result scrubber, because scrubbing its output would defeat its purpose. The allowlist is narrow and bounded to administrative tools that intentionally surface scrubbed content.

VII.J. Layer 1 (preceding boundary 1) — OS keychain credential store

In addition to the five scrubbing boundaries, the system disclosed includes an operating-system keychain credential store (preferred embodiment: `keytar` providing platform-native macOS Keychain, Windows Credential Manager, and Linux libsecret integration), accompanied by a `continuity credentials migrate` command-line interface that:

1. Scans a workspace `.env` file or equivalent in dry-run mode
2. Reports which credentials would be migrated
3. On confirmation, moves the entries to the keychain
4. Replaces the originals with references to the keychain entry

This eliminates plaintext credentials at the source and is acknowledged as an obvious application of established platform APIs.

VIII. METHOD D — MULTI-SIGNAL RELATIONSHIP INFERENCE & MARKOV CHAIN PREDICTION

VIII.A. Method summary

Two related techniques are disclosed:

1. **Multi-signal automatic inference of relationships between decision records**, combining semantic, temporal, and tag-set signals with composite confidence scoring
2. **Markov chain prediction** of next-decision tag sets from observed tag-set transitions in the decision corpus

VIII.B. Multi-signal relationship inference disclosed

Relationships between decision records are inferred **automatically**, without requiring manual annotations such as `[supersedes: ADR-001]`. The disclosed signal combination uses per-relation-type weights — different relation types weight the signals differently because the discriminating signal differs. Representative weights from the shipping code:

Relation type	Semantic	Tag overlap	File overlap	Keyword	Entity
`supersedes`	0.40	0.30	0.20	0.10	—
`causes` / `causedBy`	0.50	0.20	0.20	0.10	—
`relatedTo`	0.30	0.15	0.25	0.10	0.20

Signals used: - **Semantic similarity** — cosine similarity of the decisions' vector embeddings (computed via local MiniLM model) - **Tag-set overlap** — Jaccard similarity of decision tag sets - **File-affect overlap** — Jaccard similarity of decision `affected_files` sets - **Keyword overlap** — shared identifier-level tokens between question/answer texts - **Entity overlap** — shared named entities (file paths, identifiers, tool names) — applies to `relatedTo` only

The disclosed relationship types inferred:

Type	Trigger condition (informal)
`supersedes` / `supersededBy`	Strong semantic similarity + temporal succession + contradiction signal in answer text
`relatedTo`	Moderate semantic similarity + tag overlap, no contradiction
`causes` / `causedBy`	Temporal succession + reference in newer decision's answer to older decision's identifier
`contradicts`	

Type	Trigger condition (informal)
	Strong semantic similarity + opposite-polarity assertion in answer text
`depends_on`	Reference relationship; one decision's implementation requires another's

VIII.C. Acknowledgment versus Kantara

The closest published prior art is Kantara (Soliman, Galster, et al., 2023-2025, arxiv:2311.03358 and arxiv:2506.11005), which auto-infers similarity and contradiction between developer-rationale decisions in commit messages using DistilBERT and RoBERTa-MNLI. Distinguishing features of the disclosed method:

- **Multi-signal** versus Kantara's single-signal NLP
- **Five relation types** versus Kantara's two
- **Structured ADR log** as input versus Kantara's commit messages
- **Markov prediction layer** absent in Kantara

VIII.D. Markov chain prediction disclosed

The disclosed method models tag-set transitions across the chronological decision history as a Markov chain:

- **State space:** the set of all tag combinations observed in the decision corpus
- **Transition matrix:** `P(tag_set_t+1 | tag_set_t)` estimated from observed succession frequencies
- **Prediction:** given the current session's most recent tag set, return the highest-probability next tag set

Prediction output is consumed as a relevance signal in the retrieval ranking layer (Method A), biasing retrieval toward decisions whose tag sets are predicted to be most relevant to the current trajectory.

VIII.E. Acknowledgment versus Hindle et al.

The closest published prior art is Hindle, Barr, Gabel, Su, and Devanbu (2012), "On the Naturalness of Software" (ICSE), which applies n-gram (and equivalently, Markov) models to lexical code token autocomplete. Distinguishing feature of the disclosed method: the state space is decision tag sets (semantic categories), not lexical code tokens. The prediction output is a topic forecast for retrieval bias, not a next-token completion for autocomplete.

VIII.F. ADR-tool acknowledgment

Every Architectural Decision Record tool surveyed by the author requires **manual** relationship annotation:

- ADR-tools (Nat Pryce) — manual `Status` and `Links` fields
- MADR (Markdown Any Decision Record) — manual `Supersedes` field
- Log4brains — manual links
- Structurizr — manual relationship arrows

- Backstage ADR plugin — manual frontmatter
- ADR Manager VS Code extension — manual

Two decades of uniform manual-relationship pattern is acknowledged as the prior art baseline against which the disclosed automatic inference is measured.

IX. METHOD E — THE MEMORY AMPLIFIER THREAT MODEL EXTENSION

IX.A. Threat model summary

The disclosed threat model names and characterizes a threat class structural to memory-augmented AI assistants: a one-shot credential exposure or sensitive-content leak becomes a permanent re-exposure vector via the persistent memory layer.

The threat is distinguished from one-shot prompt injection (which is bounded in time and severity) and from attacker-injected memory poisoning (which is also documented in prior art such as OWASP ASI06:2025 and Rehberger's SpAIware September 2024). The Memory Amplifier framing inverts the attacker model: the leak originates from **legitimate** user input (paste, IDE selection, file content) rather than attacker injection, but is amplified by the same memory persistence mechanism.

IX.B. Defense disclosed

The Memory Amplifier surface is closed by:

1. **Boundary 3 + 4 scrubbing pair** (Method C): write-time AND read-time scrubbing of the memory-augmented store. The read-time layer specifically addresses credentials persisted before the write-time layer was deployed, content written via alternative paths, and tampered store content.
2. **Boundary 5 scrubbing** (Method C): MCP tool-result scrubbing prevents credentials from propagating from retrieved memory into AI context via tool responses.
3. **Governance Lock policy** (Method B): tool calls whose arguments derive from retrieved memory are evaluated under stricter source-aware policy.

IX.C. Concrete reproducer (republished)

See Section II.C above. The Atlassian token reproducer (May 2026) demonstrates the threat in production and the defense's efficacy.

IX.D. Acknowledgment of nearby prior art

- **OWASP MCP01:2025 (Token Mismanagement and Secret Exposure)** explicitly notes tokens "inadvertently stored, indexed, or retrieved later through user prompts" in MCP long-lived sessions and recommends redaction before logging. This is the closest existing framework reference. The disclosed

Memory Amplifier framing extends it by: naming the threat class as distinct, inverting the attacker model to legitimate-user-origin, and specifying dual write+read-time scrubbing as the defense.

- **OWASP ASI06:2025 (Memory & Context Poisoning)** was published December 9, 2025, after Continuity's October 31, 2025 baseline. It names persistent memory as a threat surface but from an attacker-injection perspective.
- **Mnemonic Sovereignty survey** (arxiv:2604.16548, April 2026, post-baseline) independently arrives at a write-time + retrieve-time defense framing and an "amplification through promotion" concept. Cited for completeness; published after Continuity's baseline.

X. PUBLIC DEVELOPMENT CHRONOLOGY

The following table summarizes major implementation milestones with verifiable git commit references. The full git history is publicly available at the repository hosted under `Thiagoscode/continuity`.

Date	Feature	Commit / Evidence
Oct 3, 2025	Summarized decision storage	`4713a7b`
Oct 6, 2025	File-change triggers	`a0fca93`
Oct 12, 2025	Memory ranking and token-aware compression	`a126729` / `01a4382`
Oct 14, 2025	Local embeddings (MiniLM via `@xenova/transformers`), vector store, hybrid search	`be8402d`
Oct 14-15, 2025	Decision relationship graphs and architectural intent tracking	`93cb0ef` / `0b967fd`
Oct 15, 2025	Automatic decision detection (git hook + AutoDecisionLogger)	`240fb8d`
Oct 16, 2025	Automatic SessionStart context injection	`4e30d29`
Oct 19, 2025	Raw conversation capture and analyzer	`c168fca`
Oct 21, 2025	Bounded retrieval RAG pivot (Method A)	`280ffb6`
Oct 31, 2025	**Visual Studio Code Marketplace public release**	(public release)
Nov 23, 2025	Observation layer and progressive disclosure	`9cbc113` / `f941db9`
Jan 26, 2026	Governance Lock and MCP tool interception (Method B)	`058b473`

Date	Feature	Commit / Evidence
Mar 2026	Multi-signal relationship detection refinement	(multiple commits)
Mar 2026	Markov chain prediction layer (Method D)	(commit references in public repo)
May 2026	Five-boundary credential scrubbing (Method C)	(multiple commits across `packages/core/src/security/`, `packages/mcp-server/src/middleware/`, `packages/mcp-server/src/security/`)
May 2026	`audit_secrets` MCP tool	(commit in `packages/mcp-server/src/tools/security/`)
May 2026	OS keychain credential store via `keytar`	(commit in `packages/core/src/services/CredentialStore.ts`)

Approximately 370 commits during October 2025 alone are documented in the public repository, including feature work, refactors, rollbacks, and architectural pivots — a pattern consistent with independent iterative engineering.

XI. PREFERRED EMBODIMENT REFERENCES

The preferred embodiment is implemented in the Continuity VS Code extension, with the following file locations (paths within the source repository):

Component	Source location
Shared scrub library	`packages/core/src/security/Scrubber.ts`, `packages/core/src/security/secret-patterns.ts` (27 patterns, `ENTROPY_DEFAULTS.threshold = 4.5`)
OS keychain credential store	`packages/core/src/services/CredentialStore.ts`
Boundary 1 — input scrubber	`packages/mcp-server/src/middleware/InputScrubber.ts`
Boundary 2 — AI output scrubber	`src/services/llm/LLMProviderManager.ts` chokepoint, `src/services/llm/AnthropicAPILayer.ts` chokepoint
Boundary 3 + 4 — persistence scrubbers	`src/services/DecisionCRUD.ts` write and read paths; session-notes appender; working-memory service; handoff-artifacts service

Component	Source location
Boundary 5 — MCP tool-result scrubber	`packages/mcp-server/src/security/ToolResultScrubber.ts`
`audit_secrets` tool	`packages/mcp-server/src/tools/security/definitions.ts`, `handlers.ts`; allowlisted in `SCRUB_ALLOWLIST`
Composed dispatcher path	`packages/mcp-server/src/index.ts` (composed: `scrubToolResponse` then `attachCredentialWarnings`)
Governance Lock	`src/services/governance/GovernanceLock.ts`, `src/services/governance/ToolInterceptor.ts`
Bounded retrieval	`src/services/DecisionLogger.ts` (retrieval methods), `src/services/SemanticSearchService.ts`
Markov chain prediction	`src/services/MarkovDecisionChain.ts`
Multi-signal relationship detection	`src/services/RelationshipDetector.ts`

XII. DISCLOSURE BOUNDARIES

XII.A. What this disclosure places in the public record

- The five-boundary defense-in-depth credential scrubbing architecture **as a combination**
- The specific catalog of 27 provider-specific patterns enumerated in Section VII.C
- The Shannon-entropy threshold calibration methodology of Section VII.D (the sweep evidence, not the threshold value 4.5 itself, which predates this disclosure)
- The idempotency property `scrub(scrub(x)) === scrub(x)` as a design invariant for cross-boundary scrubbing
- The `_meta.credential_warnings` MCP protocol response field shape
- The `audit_secrets` tool with self-allowlist
- The bounded retrieval architecture **as combined with** the structured Q/A/tags schema and per-record character caps on both question and answer fields
- The specific values `TOP_K_CEILING = 10`, `MAX_TEXT_LENGTH = 2,000`, `MAX_ANSWER_LENGTH = 5,000`, and the closed-form bound `T_turn ≤ 17,500 tokens`
- The runtime Governance Lock pipeline **mapped specifically to the OWASP LLM Top 10 (2025) edition numbering** across nine categories with source-aware policy
- The multi-signal relationship inference **as a four-signal weighted composition** (semantic, temporal, tag-set, file-affect) inferring five relation types

- The Markov chain prediction layer with state space = decision tag sets (distinguished from lexical-token prediction)
- The "Memory Amplifier" threat model **as a named extension** of the Compromised Context model, with attacker-model inversion (legitimate-user-origin) and dual write+read-time defense

XII.B. What this disclosure does NOT claim

- AI memory in general
- Top-K retrieval in RAG systems generally
- Runtime LLM output filtering generally
- Pre-execution AI tool-call interception generally (anticipated by Invariant Labs Apr 2025 and IsolateGPT/SecGPT Mar 2024)
- Pre-OS syscall filtering via seccomp (anticipated by IsolateGPT/SecGPT)
- Generic secret/credential scanning (anticipated by detect-secrets 2018, TruffleHog, GitGuardian, etc.)
- Regex + entropy composition for secret detection (anticipated)
- The numerical value 4.5 bits per character as an entropy threshold (anticipated by detect-secrets default)
- OS keychain integration for stored credentials (standard platform API usage)
- Architectural Decision Records in general
- Markov chain prediction in software development generally (anticipated by Hindle et al. 2012)
- Decision-relationship inference using NLP (anticipated by Kantara/Soliman et al.)

XIII. INDUSTRIAL APPLICABILITY

The disclosed methods are applicable to:

- AI coding assistants and IDE plugins with persistent memory features
- Autonomous agent frameworks with persistent scratchpads or memory stores
- RAG systems whose corpora may contain inadvertently captured credentials or other sensitive content
- Chatbot platforms offering "remember" features across sessions
- Any MCP-compliant tool ecosystem in which tool calls and tool results may carry sensitive content
- Enterprise deployments of AI assistants subject to compliance regimes that prohibit credential persistence in machine-readable logs (e.g., SOC 2, HIPAA, GDPR)
- Architectural Decision Record systems extended with automatic relationship inference

---

XIV. PUBLICATION NOTICE

This document is a public technical disclosure. It is placed in the prior-art record on the date stated above. The author retains copyright in this document. Any party may reference this disclosure in patent prosecution, prior-art searches, or technical

literature. Any party seeking to file a patent application covering the specific combinations disclosed herein is hereby on notice that this disclosure constitutes prior art.

The source code implementing the disclosed methods remains proprietary under separate license. This disclosure covers METHODS and ARCHITECTURES; it does not transfer rights in the underlying software implementation.

The author may be contacted at the address registered for the Visual Studio Code Marketplace publisher `hackerware`.

XV. REFERENCES (NON-EXHAUSTIVE)

Primary published prior art acknowledged

1. Greshake, K., et al. (2023). “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.” arXiv:2302.12173.
2. Hindle, A., Barr, E. T., Gabel, M., Su, Z., Devanbu, P. (2012). “On the Naturalness of Software.” Proceedings of ICSE 2012.
3. Soliman, M., et al. (2023, 2025). “Kantara: Auto-inference of decision relationships from commit messages.” arXiv:2311.03358 and arXiv:2506.11005.
4. Packer, C., et al. (2023). “MemGPT: Towards LLMs as Operating Systems.” arXiv:2310.08560.
5. Mem0 team (2025). “Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory.” arXiv:2504.19413.
6. Rehberger, J. (September 2024). “SpAIware: Spyware Attacks on AI Assistants.” Public writeup.
7. OWASP Foundation (2025). “OWASP Top 10 for Large Language Model Applications, 2025 edition.” genai.owasp.org.
8. OWASP Foundation (December 2025). “OWASP Agentic Security Initiative Top 10 (ASI06:2025 — Memory & Context Poisoning).”
9. OWASP Foundation (2025). “OWASP MCP Security Top 10 (MCP01:2025 — Token Mismanagement and Secret Exposure).”
10. ArXiv:2511.20920 (November 2025). “Securing the Model Context Protocol.”

Commercial / open-source systems acknowledged

1. Yelp, Inc. (2018-present). `detect-secrets` library. github.com/Yelp/detect-secrets. Default `Base64HighEntropyString.entropy_limit = 4.5`.
2. TruffleHog. github.com/trufflesecurity/trufflehog.
3. GitGuardian. gitguardian.com.
4. Invariant Labs (April 2025). Guardrails. Acquired by Snyk June 2025.
5. NVIDIA (2023). NeMo Guardrails.
6. AWS (2024). Bedrock Guardrails.
7. Microsoft Azure (2024). Prompt Shields.
8. Anthropic. Constitutional AI.
9. Lakera. Lakera Guard.
10. Cloudflare (2025). AI Gateway Guardrails.
11. OpenAI (March 2025). Agents SDK tripwires.

12. Mark, A., et al. (March 2024). “IsolateGPT / SecGPT: An execution isolation architecture for LLM-based agents.”
13. ADR-tools (Nat Pryce). github.com/npryce/adr-tools.
14. MADR. adr.github.io/madr.
15. Log4brains. github.com/thomvaill/log4brains.
16. Structurizr. structurizr.com.
17. Laiyer / Protect AI. LLM Guard.
18. CyberArk. US Patent 11,550,569 B2 (granted January 2023).

Continuity public repository

1. `Thiagoscode/continuity` on GitHub — public mirror with full development history, ~370 October 2025 commits.

END OF DEFENSIVE TECHNICAL DISCLOSURE — Version 2.0

Document SHA-256 of this file (computed at time of GitHub commit) will appear in the corresponding release tag.